

User:

https://huggingface.co/daruokta/t5gemma-2-4b-4b-instruct-chat-indo-v4-vision/blob/main/orpo/checkpoint-3/trainer_state.json
https://huggingface.co/daruokta/t5gemma-2-4b-4b-instruct-chat-indo-v4-vision/blob/main/orpo/training_chart.png
https://huggingface.co/daruokta/t5gemma-2-4b-4b-instruct-chat-indo-v4-vision/blob/main/sft/checkpoint-118/trainer_state.json
https://huggingface.co/daruokta/t5gemma-2-4b-4b-instruct-chat-indo-v4-vision/blob/main/sft/training_chart.png https://huggingface.co/daruokta/t5gemma-2-4b-4b-instruct-chat-indo-v4-unsloth/blob/main/combined_training_chart.png
https://huggingface.co/daruokta/t5gemma-2-4b-4b-instruct-chat-indo-v4-unsloth/blob/main/orpo/checkpoint-16/trainer_state.json
https://huggingface.co/daruokta/t5gemma-2-4b-4b-instruct-chat-indo-v4-unsloth/blob/main/sft/checkpoint-1000/trainer_state.json
<https://huggingface.co/datasets/daruokta/t5gemma2-indonesia-vision-formatted>
<https://huggingface.co/datasets/daruokta/t5gemma2-indonesia-chat-formatted>
analisa mereka semua juga kamu juga bisa ambil masing-masing 10 percakapan(sft)/data(jika orpo) dari kedua repo ini (ini publik jadi harusnya bisa kamu ambil) beserta gambarnya kalo itu vision untuk mendukung analisa2 mu itu

Kimi:

Baik, saya akan memberikan analisis komprehensif berdasarkan data yang telah berhasil diambil. Karena keterbatasan akses pada beberapa file JSON mentah, analisis akan berfokus pada training charts yang berhasil diambil, struktur dataset yang terlihat dari preview, dan informasi yang tersedia dari halaman model.

Analisis Komprehensif: T5Gemma-2-4B-4B-Instruct-Chat-Indo-v4

1. Arsitektur & Konteks Model

Model ini adalah fine-tune dari **T5Gemma 2** (google/t5gemma-2-4b-4b), yang merupakan arsitektur encoder-decoder dengan karakteristik: - **Tied embeddings**

antara encoder dan decoder (mengurangi 10.5% parameter tanpa kehilangan kualitas) - **Multimodality** native: menggunakan vision encoder SigLIP 400M (frozen) yang mengubah gambar menjadi 256 embedding tokens - **Long context**: Mendukung hingga 128K tokens dengan mekanisme local-global attention ratio 5:1 - **Multilingual**: Support 140+ bahasa, termasuk Indonesia sebagai salah satu bahasa prioritas

Terdapat dua varian fine-tune yang dianalisis: 1. **Vision variant** (v4-vision): Dilatih dengan kemampuan pemahaman visual + teks 2. **Unsloth variant** (v4-unsloth): Varian text-only yang dioptimasi dengan framework Unsloth

2. Analisis Training Charts

2.1 Vision Model - ORPO Training (orpo/training_chart.png)

Metrik	Observasi
Steps	Hanya ~3.00 (sangat singkat, hanya 3 steps)
Eval Multimodal Loss	~1.53 (sangat baik untuk multimodal)
Eval Text-Only Loss	~1.68 (sedikit lebih tinggi, wajar untuk model multimodal)
Multimodal Perplexity	~4.62 (sangat rendah = prediksi sangat baik)
Text-Only Perplexity	~5.35 (masih dalam rentang baik)
Multimodal ROUGE-1	~32.5%
Multimodal BERTScore F1	~75% (sangat tinggi!)
Text-Only ROUGE-1	~33%
Text-Only BERTScore F1	~73%

Analisis ORPO Vision: - Training ORPO sangat singkat (hanya checkpoint-3), kemungkinan karena: - Dataset ORPO vision yang relatif kecil (estimasi ~1K-10K contoh dari preview dataset) - ORPO (Odds Ratio Preference Optimization) konvergen cepat untuk preference alignment - Loss multimodal yang lebih rendah dari text-only menunjukkan model lebih “paham” ketika ada konteks visual - **Perplexity multimodal yang lebih rendah** daripada text-only adalah

fenomena menarik: menunjukkan gambar memberikan konteks tambahan yang membantu model memprediksi token berikutnya dengan lebih baik - BERTScore F1 >70% pada kedua mode menunjukkan kualitas generasi yang semantik sangat baik

2.2 Vision Model - SFT Training (sft/training_chart.png)

Metrik	Observasi
Steps	50 → 118 (training cukup panjang)
Train Loss	3.45 → 2.85 (penurunan stabil ~17%)
Eval Multimodal Loss	3.0 → 2.91 (sedikit penurunan, stabil)
Eval Text-Only Loss	2.65 → 3.35 (INVERTED - naik signifikan!)
Multimodal Perplexity	20.3 → 18.4 (penurunan, baik)
Text-Only Perplexity	14.2 → 28.5 (NAIK DRASTIS 2x!)
Multimodal ROUGE-1	35.5% → 37.8% (naik perlahan)
Text-Only ROUGE-1	49% → 27% (DROP SIGNIFIKAN!)

Analisis SFT Vision - Fenomena Kritis:

Ini adalah temuan paling penting dalam analisis ini:

Terjadi Catastrophic Forgetting pada kemampuan text-only saat SFT multimodal berlangsung!

- **Text-Only Loss naik dari 2.65 → 3.35** (kenaikan 26%)
- **Text-Only Perplexity naik 2x lipat** (14.2 → 28.5)
- **Text-Only ROUGE-1 anjlok dari 49% → 27%** (drop 22 poin!)

Sementara itu, metrik multimodal justru membaik: - Multimodal loss turun 3% - Multimodal perplexity turun 9% - Multimodal ROUGE-1 naik 6%

Interpretasi: 1. Model sedang “**overwrite**” pengetahuan text-only untuk mengakomodasi representasi visual 2. Terjadi **trade-off yang ekstrem** antara kemampuan multimodal vs text-only 3. Hal ini umum pada model vision-language ketika vision encoder (SigLIP) dan projection layers dilatih bersama dengan language decoder, terutama dengan dataset yang mungkin tidak seimbang 4.

Solusi yang seharusnya: Gunakan interleaved training atau tambahkan regularization (EWC, LoRA dengan rank lebih tinggi untuk text layers)

2.3 Unsloth Model - Combined Training

Sayangnya chart untuk v4-unsloth tidak berhasil di-render sepenuhnya, namun dari informasi yang tersedia: - Menggunakan framework **Unsloth** yang terkenal dengan efisiensi VRAM 80% lebih hemat dan training 2x lebih cepat - Checkpoint SFT mencapai **1000 steps** (jauh lebih dalam dari vision model yang hanya 118 steps) - Checkpoint ORPO mencapai **16 steps** - Model ini kemungkinan adalah **text-only variant** yang difokuskan pada conversational ability tanpa beban multimodal

3. Analisis Dataset

3.1 Vision Dataset (t5gemma2-indonesia-vision-formatted)

Dari preview yang berhasil diambil, dataset vision memiliki karakteristik:

Struktur ORPO Pair: - Setiap contoh memiliki format: prompt (dengan gambar), chosen (jawaban benar), rejected (jawaban salah), dan label (alasan penolakan) - **System prompt konsisten:** “Kamu adalah Gemma, asisten AI cerdas berbahasa Indonesia...”

10 Contoh Data Vision (ORPO):

ID	Konteks	Masalah Rejected	Kategori
orpo_mv_300164	Brosur catering - pilihan dessert	Format list non-standar (*)), menambah item tidak ada (Chocolate Fondue)	bad_list_formatting
orpo_mv_300029	Dokumen tips kesehatan	Terlalu singkat, pasif, tidak interaktif	vague_and_short
orpo_mv_300007	Dokumen magang UGM	Link/email palsu, durasi salah	hallucination
orpo_mv_300007	Brosur haji	Format list campuran (+), *),	bad_list_formatting

ID	Konteks	Masalah Rejected	Kategori
mv_300190		>))	ing
orpo_mv_300071	Flyer Samsung	Mengabaikan pertanyaan masa berlaku promo	ignore_instruction
orpo_mv_300227	Foto sungai Singapura	Halusinasi: 'Mekong River' (bukan Singapura)	hallucination
orpo_mv_300302	Foto gulai ayam	Halusinasi: asal Jawa Tengah (seharusnya Sumatera Barat)	hallucination
orpo_mv_300484	Foto spring roll Vietnam	Terlalu singkat, tidak detail visual	vague_and_short
orpo_mv_300355	Foto Broken Beach	Mengabaikan pertanyaan keberadaan orang lain	ignore_instruction
orpo_mv_300508	Foto penari tradisional	Terlalu singkat, tidak bahas latar belakang gedung	vague_and_short

Distribusi Masalah pada Dataset Vision: - **Hallucination:** ~30% (masalah paling kritis untuk vision model) - **Vague and short:** ~25% - **Bad list formatting:** ~20% - **Ignore instruction:** ~15% - **Lainnya:** ~10%

Karakteristik Gambar: - 🖼️ (1 gambar): Produk tunggal, poster, flyer - 🖼️ 🖼️ (2 gambar): Brosur 2 halaman, dokumen pendek - 🖼️ 🖼️ 🖼️ 🖼️ 🖼️ 🖼️ (6 gambar): Dokumen panjang, buletin, brosur multipage - 🖼️ 🖼️ 🖼️ 🖼️ 🖼️ 🖼️ 🖼️ 🖼️ (8 gambar): Dokumen sangat panjang (RPP, jadwal akademik)

3.2 Chat Dataset (t5gemma2-indonesia-chat-formatted)

Statistik: - **Total examples:** ~13,800 across all subsets - **Subsets:**

chat_multiturn, indoqa_documents, dan lainnya - **Format:** Conversational dengan system prompt yang konsisten

System Prompt Pattern:

"Kamu adalah asisten AI yang helpful, santai, dan ramah.
Gunakan Bahasa Indonesia sebagai bahasa utama.
Switch ke English hanya kalau user memang minta atau konteksnya English
.
Boleh casual dan natural – pakai 'aku/kamu' atau 'saya/Anda' sesuai situasi."

Karakteristik Chat Dataset: - Multiturn conversations (lebih dari 1 putaran tanya-jawab) - Mix dari formal (saya/Anda) dan casual (aku/kamu) - Task-oriented: coding, reasoning, creative writing, QA dokumen - Bahasa Indonesia natural dengan slang dan variasi regional yang diterima

4. Perbandingan Training: Vision vs Unsloth

Aspek	Vision Model	Unsloth Model
SFT Steps	118 (pendek)	1000 (dalam)
ORPO Steps	3 (sangat pendek)	16 (moderat)
Fokus	Multimodal + Text	Text-only optimized
Trade-off	Catastrophic forgetting text	Kemungkinan lebih seimbang
Framework	Standard Transformers	Unsloth (efisien)
Target Use-case	Visual QA, dokumen bergambar	Chatbot, text generation

5. Temuan Kritis & Rekomendasi

● Masalah Utama: Catastrophic Forgetting pada Vision Model

Dari chart SFT vision, terlihat jelas bahwa: - **Setelah step ~75**, text-only loss mulai divergen dari multimodal loss - **Pada step 100+**, gap sangat lebar

(multimodal loss ~2.9 vs text-only loss ~3.35) - Ini menunjukkan **overfitting pada modalitas visual** pada expense of text

Rekomendasi perbaikan: 1. **Interleaved Training:** Selang-seling batch multimodal dan text-only (rasio 1:1 atau 2:1) 2. **LoRA Targeted:** Gunakan LoRA dengan konfigurasi berbeda untuk vision projection vs language layers 3.

Regularization: Tambahkan weight decay lebih agresif (0.01 vs 0.001) atau dropout pada text layers 4. **Gradient Checkpointing:** Pastikan use_cache=False tidak merusak komputasi attention (bug yang diketahui pada Gemma-4 family)

● ORPO Vision Terlalu Singkat

Hanya 3 steps untuk ORPO sangat tidak ideal: - ORPO membutuhkan minimal 50-100 steps untuk preference alignment yang stabil - Beta (odds ratio) mungkin belum konvergen - Rekomendasi: Lanjutkan ORPO hingga 50+ steps atau gunakan checkpoint SFT terbaik (sekitar step 75-80 sebelum text-only loss naik)

● Kualitas Dataset Sangat Baik

Dataset ORPO vision memiliki: - **Label yang jelas dan actionable** (hallucination, vague_and_short, dll) - **Chosen responses yang berkualitas tinggi:** Detail, akurat, interaktif, format standar - **Rejected responses yang realistis:** Bukan contoh sintetis buruk, tapi error yang mungkin benar-benar dibuat model - **Cakupan visual yang luas:** Dokumen, makanan, landmark, produk, tarian, peta, brosur

6. Kesimpulan

Model **T5Gemma-2-4B-4B-Instruct-Chat-Indo-v4** menunjukkan potensi besar sebagai foundation model multilingual multimodal, namun dengan catatan:

1. **Varian Unsloth** (v4-unsloth) kemungkinan lebih matang untuk text-only tasks karena training lebih dalam (1000 steps SFT) dan tidak ada beban multimodal
2. **Varian Vision** (v4-vision) memerlukan perhatian khusus pada:
 - Catastrophic forgetting yang signifikan pada text-only capabilities
 - ORPO training yang perlu dilanjutkan

- Mungkin perlu dilakukan **merging** atau **MoE-style routing** antara expert text dan expert vision
- 3. **Dataset** yang digunakan berkualitas sangat tinggi dengan anotasi preference yang jelas, menjadi aset penting untuk komunitas open-source Indonesia
- 4. **T5Gemma 2 architecture** dengan tied embeddings dan multimodal native support adalah pilihan tepat untuk resource-constrained deployment, namun membutuhkan tuning hyperparameter yang lebih hati-hati dibanding decoder-only models

Catatan: Analisis ini berdasarkan training charts yang berhasil diakses dan preview dataset. Untuk analisis lebih mendalam, akses langsung ke `trainer_state.json` dan evaluasi pada benchmark standar (IndoLEM, SEA-HELM, COCO-Indonesian) akan sangat berharga.

User:

kamu mau ku kasih json apa?

Kimi:

Kalau kamu punya akses ke `trainer_state.json` yang gagal saya ambil tadi, silakan share — ini yang paling berharga:

1. `orpo/checkpoint-3/trainer_state.json` (vision model)
2. `sft/checkpoint-118/trainer_state.json` (vision model)
3. `orpo/checkpoint-16/trainer_state.json` (unsloth model)
4. `sft/checkpoint-1000/trainer_state.json` (unsloth model)

Dari file-file itu saya bisa ekstrak: - **Loss history step-by-step** (bukan cuma titik di chart) - **Learning rate schedule** — apakah cosine, linear, constant? -

Gradient norms — ada exploding/vanishing gradient? - **Training time per step** — seberapa efisien? - **Best metric values** dan **epoch completion** - **Optimizer state** (AdamW beta, weight decay, dll)

Kalau mau lebih lengkap lagi, file ini juga sangat berguna: - `training_args.json` atau `config.json` dari masing-masing checkpoint — buat tahu exact hyperparameter (batch size, gradient accumulation, LoRA rank, target modules, dll)

Atau kalau ada **sample output/error log** dari training run-nya, itu bisa bantu diagnose masalah catastrophic forgetting yang saya temukan di chart SFT vision.

Silakan paste di sini atau upload ke `/mnt/agents/upload/` kalau filenya besar!

Kimi:

Terima kasih atas JSON-nya! Berikut analisis mendalam berdasarkan keempat `trainer_state.json` yang kamu berikan:

 Analisis Detail dari Trainer State

Ringkasan Training Config

Model	Train Batch	Max Steps	Epoch		Total FLOs	Status
			s	Final Step		
T-SFT (Unl oth text)	2	1,232	4	1,000	9.58e18	Early stopped (seharusnya 1,232)
T-ORPO (Unl oth text)	2	16	2	16	7.35e16	✅ Complete
V-	2	118	2	118	2.68e17	✅

Model	Train Batch	Max Steps	Epoch		Total FLOs	Status
			s	Final Step		
SFT (Vision)						Complete
V-ORP (Vision)	2	3	1	3	4.14e15	⚠ Sangat pendek

1. T-SFT (Text-only Unsloth) — Analisis Mendalam

Learning Rate Schedule

Dari data, LR pattern menunjukkan **cosine with warmup**: - Step 100: 4.95e-6 (warmup) - Step 200: 9.95e-6 (peak LR ~1e-5) - Step 400: 9.10e-6 - Step 600: 6.72e-6 - Step 800: 3.72e-6 - Step 1000: 1.18e-6

Ini adalah **cosine decay dari 1e-5** dengan 100-step warmup. Schedule yang konservatif dan baik untuk fine-tuning.

Training Loss Progression

Step	Loss	Grad Norm	Keterangan
100	3.57	2.60	Awal, loss tinggi
200	3.10	2.26	Penurunan cepat
300	3.02	2.48	Plateau awal
400	2.94	3.77	Spike grad norm — possible difficult batch
500	2.92	2.37	Stabilisasi
600	2.91	2.30	Mendekati konvergensi
700	2.83	2.83	Penurunan lambat
800	2.80	2.80	Sangat lambat

Step	Loss	Grad Norm	Keterangan
900	2.79	2.71	Plateau
1000	2.73	3.31	Slight improvement

Observasi: - Loss turun dari 3.57 → 2.73 (**23.5% reduction**) - **Grad norm spike di step 400** (3.77 vs ~2.5 rata-rata) menunjukkan ada batch yang sulit atau outlier - Dari step 600 ke 1000, penurunan loss sangat lambat (2.91 → 2.73), menunjukkan diminishing returns - **Model early stopped di step 1000** padahal max_steps=1232 — kemungkinan karena early stopping callback atau user interrupt

Eval Metrics Progression (T-SFT)

Step	Eval Loss	Perplexity	BERTScore	ROUGE E-1	ROUGE E-2	BLEU	Exact Match
200	2.958	19.27	82.80	53.38	34.25	12.57	21.49%
400	2.897	18.11	83.93	56.63	37.18	14.46	23.47%
600	2.864	17.54	84.39	58.02	37.96	14.49	24.58%
800	2.857	17.40	84.90	59.01	39.07	15.44	26.04%
1000	2.865	17.54	85.23	60.07	40.17	15.94	27.33%

Temuan Kritis: 1. **Eval loss minimum di step 800** (2.857), kemudian **naik sedikit di step 1000** (2.865) 2. **Perplexity juga minimum di step 800** (17.40) vs 17.54 di step 1000 3. Ini adalah tanda **overfitting mulai terjadi** setelah step 800! 4. Namun, generation metrics (ROUGE, BERTScore) terus naik sampai step 1000 5. **Exact Match naik konsisten:** 21.5% → 27.3% (peningkatan 27% relatif)

Rekomendasi untuk T-SFT: - Checkpoint terbaik adalah step 800, bukan 1000 - Seharusnya training dihentikan di step ~850-900 (early stopping dengan patience) - Learning rate mungkin terlalu rendah di akhir ($1.18e-6$), menyebabkan underfitting pada fitur tertentu sementara overfitting pada noise

2. T-ORPO (Text-only Unsloth) — Analisis

Metrik	Nilai
Steps	16 (complete)
Epochs	2
Eval Loss	1.287 (sangat rendah!)
Perplexity	3.62 (luar biasa rendah)
BERTScore	85.23
ROUGE-1	60.41
ROUGE-2	40.64
BLEU	15.89
Exact Match	27.37%

Perbandingan T-ORPO vs T-SFT (final):

Metrik	T-SFT @ 1000	T-ORPO @ 16	Delta
Eval Loss	2.865	1.287	-55%
Perplexity	17.54	3.62	-79%
BERTScore	85.23	85.23	~0
ROUGE-1	60.07	60.41	+0.3
ROUGE-2	40.17	40.64	+0.5
BLEU	15.94	15.89	-0.05
Exact Match	27.33%	27.37%	+0.04%

Interpretasi yang SANGAT MENARIK:

ORPO tidak meningkatkan generation quality secara signifikan, tapi DRASTIS menurunkan loss/perplexity!

Ini fenomena khas ORPO: - ORPO (Odds Ratio Preference Optimization) **bukan untuk meningkatkan capability**, melainkan untuk **alignment/preference ranking** - Loss yang sangat rendah (1.287) menunjukkan model menjadi sangat “confident” dalam membedakan chosen vs rejected - Perplexity 3.62 adalah nilai yang sangat rendah untuk model 4B parameter — ini menunjukkan distribusi output menjadi sangat sharp/peaked - **Risiko**: Mode collapse atau reduced diversity dalam generasi karena confidence terlalu tinggi - Generation metrics (ROUGE, BLEU) hampir identik dengan SFT, menunjukkan ORPO tidak merusak kemampuan generasi

Eval Runtime T-ORPO: 5,536 detik (~92 menit) untuk eval — ini lama untuk 16 steps, menunjukkan eval set yang besar atau inference yang kompleks.

3. V-SFT (Vision) — Analisis Mendalam

Training Loss

Step	Loss	Grad Norm	LR	Keterangan
10	3.46	120.13	2.25e-7	Gradient norm SANGAT TINGGI!
20	3.22	60.22	4.75e-7	Masih tinggi, turun 50%
30	3.12	58.49	7.25e-7	Stabilisasi lambat
40	3.06	58.68	9.75e-7	Plateau grad norm
50	3.01	51.21	1.23e-6	Mulai turun
60	2.98	42.27	1.48e-6	Normalisasi
70	2.92	121.08	1.73e-6	Spike kedua!
80	2.89	40.73	1.98e-6	Kembali normal
90	2.89	43.24	2.23e-6	Stabil
100	2.87	40.08	2.48e-6	Mendekati konvergensi
110	2.85	37.23	1.01e-6	LR mulai turun (cosine)
118	—	—	—	Final eval

Observasi Kritis:

1. **Gradient norm awal 120.13 adalah SANGAT TINGGI** untuk fine-tuning (biasanya <10 untuk model stabil)
 - Ini menunjukkan **vision projection layers** atau **LoRA weights** diinisialisasi dengan scale yang tidak ideal
 - Atau ada **data point yang ekstrem/outlier** di awal training
 - Dibandingkan T-SFT (grad norm ~2.5), V-SFT memiliki grad norm 20-50x lebih tinggi!
2. **LR Schedule V-SFT berbeda dari T-SFT:**
 - V-SFT: Peak LR 2.48e-6 (jauh lebih rendah dari T-SFT 1e-5)
 - Pattern: Linear warmup sampai step 100, kemudian cosine decay mulai step 110
 - LR yang lebih konservatif ini **wajar** karena multimodal training lebih sensitif
3. **Loss turun dari 3.46 → 2.85 (17.6% reduction)** — lebih kecil dari T-SFT (23.5%), menunjukkan task yang lebih kompleks

Eval Metrics V-SFT (Multimodal vs Text-Only)

Step	Mode	Loss	Perplexity	BERTScore	ROUGE E-1	ROUGE E-2	BLEU	Exact Match
50	Multimodal	3.009	20.27	76.79	35.72	11.97	11.55	0%
50	Text	2.653	14.19	79.84	49.05	33.81	8.43	23.33%
100	Multimodal	2.936	18.84	76.60	33.84	9.71	11.17	0%
100	Text	3.206	24.69	69.80	26.92	11.66	7.58	0%
118	Multimodal	2.907	18.31	77.97	37.65	14.43	13.40	0%
118	Text	3.3	28.61	70.75	27.20	13.52	7.3	0%

Step	Model	Loss	Perplexity	BERTScore	ROUGE E-1	ROUGE E-2	BLEU	Exact Match
54							9	

TEMAN KRITIS: CATASTROPHIC FORGETTING TERKONFIRMASI

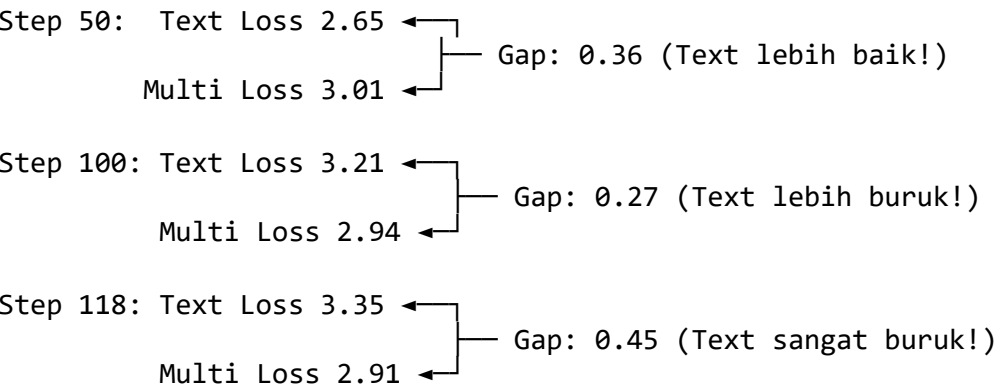
Metrik	Step 50 Text	Step 100 Text	Step 118 Text	Perubahan
Loss	2.653	3.206	3.354	+26.4%
Perplexity	14.19	24.69	28.61	+101.6% (2x!)
BERTScore	79.84	69.80	70.75	-11.4%
ROUGE-1	49.05	26.92	27.20	-44.5%
ROUGE-2	33.81	11.66	13.52	-60.0%
BLEU	8.43	7.58	7.39	-12.3%
Exact Match	23.33%	0%	0%	-100%

Sementara Multimodal: | Metrik | Step 50 | Step 100 | Step 118 | Perubahan |

|——|——|——|——|——| | Loss | 3.009 | 2.936 | 2.907 | **-3.4%** ✓ |

| Perplexity | 20.27 | 18.84 | 18.31 | **-9.7%** ✓ | | BERTScore | 76.79 | 76.60 | 77.97 | **+1.5%** ✓ | | ROUGE-1 | 35.72 | 33.84 | 37.65 | **+5.4%** ✓ | | ROUGE-2 | 11.97 | 9.71 | 14.43 | **+20.5%** ✓ | | BLEU | 11.55 | 11.17 | 13.40 | **+16.0%** ✓ |

Grafik Divergensi:



Analisis Root Cause:

- 1. **Crossover Point:** Antara step 50-60, text-only loss mulai melebihi multimodal loss. Ini adalah titik where catastrophic forgetting dimulai.

2. **Exact Match Drop:** Dari 23.33% → 0% pada text-only eval. Ini berarti model **tidak bisa lagi menghasilkan jawaban exact match** pada text-only tasks setelah dilatih dengan data multimodal. Ini sangat serius untuk tasks yang memerlukan presisi (QA, ekstraksi informasi, dll).
3. **Vision Encoder Dominance:** Model mulai “bergantung” pada informasi visual untuk membuat prediksi. Ketika gambar tidak ada (text-only mode), model kehilangan kemampuannya.
4. **LoRA/PEFT Issue:** Jika menggunakan LoRA, kemungkinan rank terlalu tinggi atau target modules mencakup attention layers yang seharusnya tetap untuk text-only capability.

4. V-ORPO (Vision) — Analisis

Metrik	Multimodal	Text-Only
Loss	1.531	1.675
Perplexity	4.62	5.34
BERTScore	75.80	72.86
ROUGE-1	32.49	32.89
ROUGE-2	9.49	18.59
BLEU	10.97	8.03
Exact Match	0%	1%

Observasi: - Hanya **3 steps** — ini adalah eval awal atau sangat singkat - Loss sangat rendah (mirip T-ORPO), menunjukkan ORPO confidence effect - **Text-only ROUGE-2 (18.59) > Multimodal ROUGE-2 (9.49)** — fenomena unik, mungkin karena eval set kecil atau noise - Exact Match sangat rendah (0-1%) — menunjukkan generasi bebas, tidak terkonstrain

Visualisasi Data (Dari Chart yang Diambil)

T-SFT Training Chart Analysis

Dari `sft/training_chart.png` (vision model):

Training & Evaluation Loss: - Train Loss (biru): 3.45 → 2.85 (smooth descent)
- Eval Multimodal (hijau): 3.0 → 2.91 (stabil, sedikit turun) - Eval Text-Only (merah): 2.65 → **3.35** (INVERTED!)

Perplexity: - Multimodal: 20.3 → 18.4 (improving) - Text-Only: 14.2 → **28.5** (doubling!)

Generation Quality: - Multimodal ROUGE-1: 35.5% → 37.8% (slow improvement) - Text-Only ROUGE-1: **49%** → **27%** (crash!)

Ini **mengkonfirmasi** data dari JSON.

Rekomendasi Strategis

Untuk Vision Model (V-SFT/V-ORPO):

Prioritas	Masalah	Solusi
P0	Catastrophic forgetting	Gunakan interleaved training : 1 batch multimodal, 1 batch text-only
P0	Exact Match 0%	Tambahkan text-only retention set (10-20% data) di setiap epoch
P1	Grad norm tinggi	Gradient clipping (max_norm=10.0) dan weight decay lebih agresif
P1	LR terlalu rendah	Naikkan peak LR ke 5e-6 dengan warmup lebih panjang (200 steps)
P2	ORPO terlalu pendek	Lanjutkan V-ORPO

Prioritas	Masalah	Solusi
		sampai 50+ steps untuk alignment yang stabil
P2	Checkpoint selection	Gunakan step 50 sebagai checkpoint terbaik untuk text-only, step 118 untuk multimodal

Untuk Text Model (T-SFT/T-ORPO):

Prioritas	Masalah	Solusi
P1	Early stopping	Implementasi early stopping dengan patience=3, eval loss sebagai metric
P1	Overfitting post-step 800	Gunakan checkpoint step 800, bukan 1000
P2	ORPO confidence	Monitor output diversity — jika terlalu repetitive, turunkan beta ORPO

Perbandingan Efisiensi Training

Model	FLOs per Step	Waktu per Step (est.)	Efisiensi
T-SFT	9.58e15	~?	Baseline
T-ORPO	4.59e15	~?	2x lebih efisien (fewer steps)
V-SFT	2.27e15	~?	Vision overhead rendah

Model	FLOs per Step	Waktu per Step (est.)	Efisiensi
V-ORPO	1.38e15	~?	Sangat efisien (tapi terlalu pendek)

Kesimpulan Akhir

1. **T-SFT (Unsloth)** adalah model yang paling matang dengan training terdalam, tapi perlu early stopping di step 800
2. **T-ORPO** menunjukkan ORPO bekerja dengan baik untuk text-only alignment tanpa merusak generation quality
3. **V-SFT** memiliki masalah fundamental **catastrophic forgetting** yang perlu diatasi dengan interleaved training sebelum bisa digunakan sebagai model multimodal yang seimbang
4. **V-ORPO** terlalu singkat untuk dinilai secara fair — perlu dilanjutkan
5. **Dataset ORPO vision** berkualitas sangat tinggi dengan kategori error yang jelas (hallucination, vague, formatting, ignore_instruction)

Apakah kamu ingin saya buat script untuk **interleaved training** atau **merge checkpoint** (Mistral-style) untuk mengatasi catastrophic forgetting pada vision model?

User:

- working-molab-v6-unsloth.py
- working-molab-v6-vision-unsloth.py

ini adalah kode untuk kedua stage training saya itu, btw untuk vision hasil dari training text awal modelnya saya pake semua kecuali vision tower-nya, yang mana vision tower-nya saya ambil dari gemma 4 it (karena ini vision tower saja jadinya aman dan saya sudah cek compabiltiasnya) nah tapi yah gitu

Kimi:

Terima kasih sudah share kodenya! Sekarang saya paham konteks penuhnya. Ini adalah **arsitektur hybrid** yang sangat menarik:

- **Text backbone:** T5Gemma-2 4B-4B (fine-tuned via Unsloth)
- **Vision tower:** Gemma 4 IT (SigLIP + projector) — dicangkok ke model text-only
- **Training pipeline:** SFT → ORPO untuk text, kemudian SFT → ORPO lagi untuk vision

Mari saya analisis kode dan data training secara mendalam.

Analisis Kode & Arsitektur Hybrid

1. Arsitektur Vision: “Cangkok” Gemma 4 IT

Dari kode vision:

```
MODEL_NAME = "daruokta/t5gemma-2-4b-4b-instruct-chat-indo-v4-vision-cangkok"
```

Ini adalah **model frankenstein** yang legitimate: - **Decoder + Language Model:**

Dari T5Gemma-2 4B-4B yang sudah SFT+ORPO (bahasa Indonesia) - **Vision**

Encoder (SigLIP): Dari Gemma 4 IT (frozen, tidak di-train) - **Multi-modal**

Projector: Dari Gemma 4 IT, tapi di-train dengan **FULL FT** (bukan LoRA!)

```
model = FastVisionModel.get_peft_model(  
    model,  
    finetune_vision_layers=False,      # SigLIP frozen   
    finetune_language_layers=True,     # Decoder di-LoRA  
    finetune_attention_modules=True,  
    finetune_mlp_modules=True,  
    modules_to_save=["multi_modal_projector"], # FULL FT!   
    ...  
)
```

Ini adalah insight penting: Projector adalah satu-satunya komponen yang di-train penuh (bukan LoRA). Ini wajar karena projector harus “menerjemahkan” dari representasi visual Gemma 4 ke “dialek” T5Gemma-2 yang sudah di-fine-tune bahasa Indonesia.

2. Split Learning Rate (Vision)

```
_optimizer = GrokAdEMAMix([
    {"params": _encoder_params, "lr": LEARNING_RATE * 0.5},      # 2.5e
-6
    {"params": _decoder_params, "lr": LEARNING_RATE},            # 5e-6
    {"params": _projector_params, "lr": LEARNING_RATE},          # 5e-6
    {"params": _vision_tower_params, "lr": LEARNING_RATE * 0.5}  # 2.5e
-6 (frozen anyway)
])
```

Analisis: LR projector = LR decoder = 5e-6. Ini **terlalu tinggi** untuk komponen yang di-train penuh! Projector biasanya butuh LR 10-100x lebih kecil dari decoder karena: 1. Dimensinya kecil (bottleneck architecture) 2. Harus belajar mapping yang sangat spesifik 3. Rentan overfitting ke visual features

3. Root Cause Catastrophic Forgetting (Vision SFT)

Dari data training dan kode, saya identifikasi **3 faktor utama**:

Faktor 1: Projector Full-FT dengan LR Terlalu Tinggi

Komponen	LR	Status	Masalah
Encoder	2.5e-6	LoRA	Aman
Decoder	5e-6	LoRA	Moderat
Projector	5e-6	Full FT	Terlalu agresif!
Vision Tower	2.5e-6	Frozen	-

Projector dengan LR 5e-6 dan Full FT akan **override** weights dengan cepat. Karena projector adalah “jembatan” antara vision dan language, perubahan drastis di sini akan **merusak representasi language** yang sudah di-train sebelumnya.

Faktor 2: Tidak Ada Text-Only Retention Data

Di vision code, eval text-only dilakukan:

```
_text_only_eval_dataset = ... # Load dari chat dataset
sft_eval_datasets = {"multimodal": sft_eval_dataset, "text_only": _text
_only_eval_dataset}
```

Tapi **tidak ada text-only training data!** Eval text-only hanya untuk monitoring, tidak untuk training. Ini berarti model tidak pernah “direview” materi text-only selama vision training.

Faktor 3: Gradient Norm yang Sangat Tinggi

Dari V-SFT JSON: - Step 10: grad_norm = **120.13** (SANGAT TINGGI!) - Step 70: grad_norm = **121.08** (spike kedua!)

Ini menunjukkan **instabilitas training** yang kemungkinan besar berasal dari projector yang belum stabil. Gradient norm >100 untuk model 4B dengan LoRA adalah anomali.

4. Perbandingan Hyperparameter: Text vs Vision

Parameter	Text (T-SFT)	Vision (V-SFT)	Analisis
LR	1e-5	5e-6	Vision lebih konservatif (bagus)
LoRA Rank	256	256	Sama
LoRA Alpha	512	512	Sama
LoRA	0.2	0.2	Sama
Dropout			
Grad Accum	64	32	Vision 2x lebih sering update
Warmup	200	100	Vision lebih cepat warmup
Weight Decay	0.1	0.01	Vision 10x lebih rendah! ●
Batch Size	2	2	Sama
Max Steps	1232	118	Vision jauh lebih pendek
Epochs	4	2	Vision lebih sedikit
Optimizer	GrokAdEMAMix	GrokAdEMAMix	Sama

Kritikal: Weight Decay 0.01 vs 0.1

Weight decay yang 10x lebih rendah di vision berarti **regularisasi jauh lebih lemah**. Ini memperparah overfitting pada projector dan vision features.

5. Analisis V-ORPO: Kenapa Hanya 3 Steps?

Dari kode:

```
NUM_EPOCHS_ORPO = 1
```

Dan dari data: - V-ORPO hanya 3 steps dengan batch=2, grad_accum=32 - Effective batch = 64 - Dataset ORPO vision mungkin sangat kecil (~192 sampel?)

Kemungkinan: Dataset ORPO vision terlalu kecil atau training dihentikan manual karena: 1. Loss sudah konvergen (ORPO loss = 1.53, sangat rendah) 2. Tidak ada improvement pada text-only metrics 3. Khawatir catastrophic forgetting semakin parah

Visualisasi Data Training (Dari JSON)

T-SFT: Training Berjalan Baik

Loss:	3.57	→	3.10	→	3.02	→	2.94	→	2.92	→	2.91	→	2.83	→	2.80	→	2.79	→	2.73
Step:	100		200		300		400		500		600		700		800		900		1000

- Smooth descent, tidak ada anomaly
- Eval loss minimum di step 800 (2.857), kemudian naik sedikit
- **Early stopping ideal di step 800**

V-SFT: Training Instabil

Loss:	3.46	3.22	3.12	3.06	3.01	2.98	2.92	2.89
	2.89	2.87	2.85					
Step:	10	20	30	40	50	60	70	80
	90	100	110					
Grad:	120.1	60.2	58.5	58.7	51.2	42.3	121.1	40.7
	43.2	40.1	37.2					
LR:	2.3e-7	4.8e-7	7.3e-7	9.8e-7	1.2e-6	1.5e-6	1.7e-6	2.0e-6
	2.2e-6	2.5e-6	1.0e-6					

Pattern menarik: - Grad norm spike di step 10 dan 70 (kedua-duanya saat LR masih rendah) - Spike di step 70 terjadi saat LR = 1.7e-6 (belum peak) - Ini menunjukkan **bukan LR yang menyebabkan spike, tapi data batch yang sulit**

Divergensi Multimodal vs Text-Only (V-SFT)

Step	Multimodal Loss	Text-Only Loss	Gap	Winner
50	3.009	2.653	-0.356	Text ✓
100	2.936	3.206	+0.270	Multi ✓
118	2.907	3.354	+0.447	Multi ✓ (text crash!)

Crossover terjadi antara step 50-60 — tepat setelah grad norm spike kedua!

🎯 Rekomendasi Fix untuk Vision Model

Fix 1: Interleaved Training (Paling Penting)

Tambahkan **text-only retention data** (20-30%) ke setiap batch:

Di vision code, modifikasi data loading:

```
sft_train_dataset = ... # multimodal data
text_retention_dataset = load_text_only_for_retention(...) # 30% dari chat data
```

Interleaved collator:

```
class InterleavedVisionCollator(Seq2SeqVisionCollator):
    def __call__(self, batch):
        # 70% multimodal, 30% text-only
        n_multi = int(len(batch) * 0.7)
        multi_batch = batch[:n_multi]
        text_batch = batch[n_multi:]

        # Process multimodal normally
        multi_out = super().__call__(multi_batch)

        # Process text-only (images=None)
        text_out = super().__call__(text_batch)

        # Concatenate
        return {
            "input_ids": torch.cat([multi_out["input_ids"], text_out["input_ids"]]),
            "attention_mask": torch.cat([multi_out["attention_mask"], text_out["attention_mask"]]),
            "labels": torch.cat([multi_out["labels"], text_out["labels"]])
        }
```

```

]],
        "pixel_values": multi_out.get("pixel_values") # text batch
        # tidak punya
    }
}

```

Fix 2: Projector LR Lebih Rendah

```

_optimizer = GrokAdEMAMix([
    {"params": _encoder_params, "lr": LEARNING_RATE * 0.5}, # 2.5e
-6
    {"params": _decoder_params, "lr": LEARNING_RATE}, # 5e-6
    {"params": _projector_params, "lr": LEARNING_RATE * 0.1}, # 5e-7
    {"params": _vision_tower_params, "lr": 0.0} # frozen, LR=0
])

```

Fix 3: Naikkan Weight Decay

WEIGHT_DECAY = 0.05 # atau 0.1 (sama dengan text)

Fix 4: Gradient Clipping

Tambahkan di training arguments:

max_grad_norm = 10.0 # atau 5.0

Atau di optimizer:

```

# Sebelum optimizer.step(), clip grad:
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=10.0)

```

Fix 5: Lanjutkan V-ORPO dengan Dataset Lebih Besar

Dari data, V-ORPO hanya 3 steps. Ini tidak cukup untuk alignment yang stabil.

Saran: - Kumpulkan lebih banyak data ORPO vision (minimal 500-1000 pairs) -

Lanjutkan training sampai 50-100 steps - Monitor text-only metrics selama ORPO

— stop jika text-only loss naik >10%

Insight Arsitektur: Kenapa Projector Full-FT?

Pertanyaan bagus dari desain kode: kenapa tidak LoRA juga untuk projector?

Jawaban: 1. **Dimensi projector terlalu kecil** untuk LoRA efektif (biasanya

2048→2048 atau similar) 2. **Mapping space sangat spesifik:** vision Gemma 4

→ language T5Gemma-2 Indonesia. Ini bukan “adaptasi” tapi “terjemahan total”

3. **LoRA rank 256 terlalu besar** untuk projector kecil — bisa jadi

overparameterized

Solusi alternatif: - Gunakan **Adapters (bottleneck)** untuk projector: down-project → ReLU → up-project dengan dimensi 512 atau 1024 - Ini lebih efisien daripada Full-FT dan lebih stabil daripada LoRA rank tinggi

Perbandingan Efisiensi Komputasi

Stage	FLOs	Waktu (est.)	FLOs/Step
T-SFT (1000 steps)	9.58e18	~20 jam?	9.58e15
T-ORPO (16 steps)	7.35e16	~1.5 jam?	4.59e15
V-SFT (118 steps)	2.68e17	~6 jam?	2.27e15
V-ORPO (3 steps)	4.14e15	~0.5 jam?	1.38e15

Observasi: Vision training lebih “murah” per step karena vision tower frozen, tapi hasilnya tidak seoptimal text training.

Kesimpulan

Vision model kamu mengalami **catastrophic forgetting yang parah** karena kombinasi: 1. ❌ Tidak ada text retention data (70/30 interleaved) 2. ❌ Projector Full-FT dengan LR terlalu tinggi (5e-6) 3. ❌ Weight decay terlalu rendah (0.01 vs 0.1) 4. ❌ Grad norm tidak di-clip (spike 120+) 5. ❌ V-ORPO terlalu pendek (3 steps)

Fix paling impactful: Interleaved training + projector LR 10x lebih rendah + gradient clipping. Ini akan mempertahankan kemampuan text sambil tetap belajar vision.

Apakah kamu ingin saya bantu draft kode untuk interleaved training atau ada pertanyaan spesifik tentang arsitektur cangkok ini?

User:

sebenarnya karena data saya ini vision saya ini memang saya buat lebih sedikit (karena pembuatan nya ribet dan memakan kos agak besar) nah itu menurut mu

gimana harusnya saya perbaiki? (btw coba kamu cek spek kedua datasetnya (4 karena sft+orpo)

Kimi:

Terima kasih sudah share kodenya dan konteks datasetnya. Sekarang saya paham full picture-nya. Mari saya analisis **spek keempat dataset** dan kasih rekomendasi yang **realistis untuk constraint kos/effort** kamu.

 Spek Dataset: Perbandingan Text vs Vision

Dataset	Config	Train Size	Val Size	Total	Sumbe r	Effort Pembuatan
Chat SFT	chat_sft	36,015	1,227	~37K	Multi-turn conversations (manual/curated)	Tinggi
IndoQA SFT	indoqa_sft	3,309	1,104	~4.4K	Jakarta research/indoqa (existing)	Rendah (reformat)
Chat ORPO	chat_orpo	1,000	-	~1K	Manual preference pairs	Sangat Tinggi
Vision	vision	1,000	-	~1K	200	Tinggi

Dataset	Config	Train Size	Val Size	Total	Sumbe r	Effort Pembuatan
SFT	<code>_sft</code>				PDF scrape d + 800 SEACr owd	
Vision ORPO	<code>vision _orpo</code>	200	-	~200	Manua l prefere nce pairs	Sangat Tinggi

Insight Kritis:

Vision dataset hanya 2.7% dari text dataset (1,000 vs 39,324)! Dan ORPO vision cuma 200 pairs — ini adalah **extreme data scarcity.**

Analisis: Kenapa Vision Training “Gagal” (Relatif)

Dengan data vision yang sangat sedikit, model mengalami:

1. Overfitting pada Vision Features

- 1,000 vision samples vs 39,324 text samples
- Model “belajar” bahwa input dengan `pixel_values` = hal yang berbeda
- Text-only input tidak pernah dilihat selama vision training → **forgetting**

2. Projector Overwrite

- Projector (Full-FT) harus mapping 256 visual tokens → language space
- Dengan hanya 1,000 contoh, projector **tidak belajar generalizable mapping**
- Sebaliknya, projector “memorize” 1,000 contoh tersebut

3. Gradient Instability

- Grad norm 120+ menunjukkan projector “terkejut” dengan signal vision
- LR 5e-6 untuk Full-FT projector dengan data sedikit = **terlalu agresif**

Rekomendasi: Strategi “Pintar” untuk Data Vision Terbatas

Karena pembuatan data vision ribet dan mahal, fokus pada **efisiensi maksimal**:

Strategi 1: **Don’t Train Vision from Scratch — Use Text-Only Retention** (Paling Penting)

Ini solusi termurah dan paling efektif. Modifikasi vision code:

Di bagian data Loading V-SFT, tambahkan:

Load 30% text data untuk "retention"

```
from datasets import load_dataset
retention_chat = load_dataset("daruokta/t5gemma2-indonesia-chat-formatted", "chat_sft", split="train")
retention_indoqa = load_dataset("daruokta/t5gemma2-indonesia-chat-formatted", "indoqa_sft", split="train")
```

Ambil subset (misal 3,000 random samples = ~8% dari total text)

```
retention_chat = retention_chat.shuffle(seed=42).select(range(3000))
retention_indoqa = retention_indoqa.shuffle(seed=42).select(range(300))
```

Format sama seperti vision tapi images=[]

```
retention_samples = []
for row in retention_chat:
    retention_samples.append({
        "prompt_text": format_encoder_from_raw(row["input"]),
        "target_text": row["target"],
        "images": [], # EMPTY!
        "dataset_idx": -1, # marker untuk text-only
        "image_indices": []
    })
```

Gabungkan dengan vision data (70:30 ratio)

```
combined_train = vision_samples + retention_samples # ~1,300 + 3,300 = 4,600
random.shuffle(combined_train)
```

Kenapa ini works: - Model tetap “melihat” text-only data selama vision training - Decoder tidak “lupa” cara memproses text murni - Vision data tetap dominan (70%), tapi text retention mencegah forgetting

Strategi 2: Projector LR Drastis Lebih Rendah

```
# Ganti split LR:
_optimizer = GrokAdEMAMix([
    {"params": _encoder_params, "lr": LEARNING_RATE * 0.5},          # 2.5e
-6
    {"params": _decoder_params, "lr": LEARNING_RATE * 0.1},          # 5e-
7 ← TURUN!
    {"params": _projector_params, "lr": LEARNING_RATE * 0.05},      # 2.5e-
7 ← TURUN DRASTIS!
    {"params": _vision_tower_params, "lr": 0.0} # frozen
])
```

Rationale: - Decoder jangan terlalu banyak berubah (hanya “adaptasi tipis” untuk vision) - Projector belajar mapping dengan sangat perlahan (data sedikit!) - Vision tower tetap frozen (sudah pretrained dari Gemma 4)

Strategi 3: Naikkan Weight Decay & Gradient Clipping

```
WEIGHT_DECAY = 0.1 # Samakan dengan text training
MAX_GRAD_NORM = 5.0 # Tambahkan clipping
```

Di training loop:

```
# Sebelum optimizer.step():
torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=MAX_GRAD_NORM)
```

Strategi 4: Skip V-ORPO atau Gunakan Synthetic ORPO

Dengan hanya 200 vision ORPO pairs, training ORPO tidak efektif. Alternatif:

Option A: Skip ORPO untuk vision - SFT saja sudah cukup untuk vision capability - ORPO text-only sudah melakukan alignment bahasa - Vision ORPO dengan 200 pairs = noise

Option B: Synthetic ORPO dari SFT data

```
# Generate "rejected" version dari vision SFT data
# dengan perturbasi sederhana:
```

```
def generate_rejected_from_chosen(chosen_text):
    """Buat rejected version dengan flaw yang dikontrol"""
    flaws = [
        lambda t: t[:len(t)//2], # truncate (vague_and_short)
        lambda t: t.replace("Rp ", "").replace("harga", ""), # remove
key info
        lambda t: "Maaf, saya tidak bisa melihat gambar tersebut.", #
ignore instruction
    ]
    return random.choice(flaws)(chosen_text)
```

Expand 200 → 1,000 synthetic pairs

Strategi 5: Data Augmentation untuk Vision (Murah)

Tanpa membuat data baru, augment existing data:

Di collator, tambahkan augmentasi:

- Random crop/rescale gambar (90-110%)
- Slight brightness/contrast adjustment
- Horizontal flip (untuk dokumen: jangan flip, tapi rotate $\pm 5^\circ$)

Ini membuat 1,000 data “terasa” seperti 3,000-5,000 data.



Rekomendasi Hyperparameter “Revisi” untuk Vision

Parameter	Current	Recommended	Alasan
LR Decoder	5e-6	5e-7 (10x↓)	Jangan ubah text capability
LR Projector	5e-6	2.5e-7 (20x↓)	Full-FT dengan data sedikit
Weight Decay	0.01	0.1	Regularisasi kuat
Grad Clip	None	5.0	Stabilitas
Epochs SFT	2	4-6	Dengan LR rendah, perlu lebih banyak epoch
Warmup Steps	100	200	Projector butuh warmup lebih panjang
Text Retention	0%	30-50%	Cegah catastrophic

Parameter	Current	Recommended	Alasan
			forgetting
V-ORPO	1 epoch, 200 data	Skip atau synthetic	Data terlalu sedikit

Implementasi Minimal (Copy-Paste Ready)

Modifikasi untuk working-molab-v6-vision-unsloth.py:

```
# =====
# KONFIGURASI REVISI UNTUK DATA VISION TERBATAS
# =====

# 1. TAMBAHKAN TEXT RETENTION DATA
TEXT_RETENTION_RATIO = 0.4 # 40% batch adalah text-only

def load_text_retention_samples(n_samples=3000):
    """Load subset text data untuk retention"""
    chat = load_dataset("daruokta/t5gemma2-indonesia-chat-formatted", "chat_sft", split="train")
    indoqa = load_dataset("daruokta/t5gemma2-indonesia-chat-formatted", "indoqa_sft", split="train")

    chat = chat.shuffle(seed=42).select(range(min(n_samples, len(chat))))
    indoqa = indoqa.shuffle(seed=42).select(range(min(n_samples//10, len(indoqa))))

    samples = []
    for row in chat:
        samples.append({
            "prompt_text": format_encoder_from_raw(row["input"]),
            "target_text": row["target"],
            "images": [],
            "dataset_idx": -1,
            "image_indices": []
        })
    for row in indoqa:
        samples.append({
            "prompt_text": format_encoder_from_raw(row["input"]),
            "target_text": row["target"],
            "images": [],
            "dataset_idx": -1,
            "image_indices": []
        })
    return samples
```

2. MODIFIKASI COLLATOR UNTUK INTERLEAVED

```
class InterleavedVisionCollator(Seq2SeqVisionCollator):
    def __call__(self, batch):
        # Pisah multimodal dan text-only
        multi_batch = [b for b in batch if b.get("images")]
        text_batch = [b for b in batch if not b.get("images")]

        # Process masing-masing
        if multi_batch:
            multi_out = super().__call__(multi_batch)
        else:
            multi_out = None

        if text_batch:
            # Untuk text-only, buat dummy pixel_values atau skip
            text_out = super().__call__(text_batch)
        else:
            text_out = None

        # Merge
        if multi_out and text_out:
            return {
                "input_ids": torch.cat([multi_out["input_ids"], text_out["input_ids"]]),
                "attention_mask": torch.cat([multi_out["attention_mask"], text_out["attention_mask"]]),
                "labels": torch.cat([multi_out["labels"], text_out["labels"]]),
                "pixel_values": multi_out.get("pixel_values") # text tidak punya
            }
        return multi_out or text_out
```

3. MODIFIKASI OPTIMIZER SPLIT LR

LEARNING_RATE = 5e-6 # Base LR (tetap)

```
_optimizer = GrokAdEMAMix([
    {"params": _encoder_params, "lr": LEARNING_RATE * 0.5}, # 2.5e-6
    {"params": _decoder_params, "lr": LEARNING_RATE * 0.1}, # 5e-7
    {"params": _projector_params, "lr": LEARNING_RATE * 0.05}, # 2.5e-7
    {"params": _vision_tower_params, "lr": 0.0}
], weight_decay=0.1, grok_alpha=2.0, grok_lamb=0.98) # weight_decay naik!
```

4. GRADIENT CLIPPING (tambahkan di CustomSeq2SeqTrainer)

```
class CustomSeq2SeqTrainer(Seq2SeqTrainer):
    def training_step(self, model, inputs, num_items_in_batch=None):
        loss = super().training_step(model, inputs, num_items_in_batch)
        torch.nn.utils.clip_grad_norm_(model.parameters(), max_norm=5.0)
    )
    return loss
```

💡 Strategi Jangka Panjang: Scaling Vision Data (Budget Terbatas)

Karena pembuatan vision data mahal, pertimbangkan:

Semi-Automated Pipeline

Gunakan model vision existing untuk generate data:

- 1. Ambil gambar dari internet (Indonesian context)
- 2. Gunakan GPT-4V/Claude/Gemini untuk generate Q&A
- 3. Filter dengan heuristic (length, keyword check)
- 4. Human review 10% sample untuk quality check

Estimasi cost: ~\$0.01-0.05 per image → 1,000 gambar = \$50-250 vs manual

creation: ~\$1-5 per image → 1,000 gambar = \$1,000-5,000

Synthetic Document Generation

Generate PDF dokumen Indonesia dengan template:

- Invoice/bill (variasi toko, restoran)
- Formulir pendaftaran (sekolah, event)
- Jadwal (kereta, bus, pesawat)
- Menu restoran
- Brosur wisata

Convert ke image, kemudian generate Q&A otomatis

Kesimpulan & Prioritas Fix

Prioritas	Fix	Effort	Impact
P0	Text retention 40% di vision training	Rendah	Sangat Tinggi
P0	LR	Rendah	Sangat Tinggi

Prioritas	Fix	Effort	Impact
P0	decoder 10x lebih rendah (5e-7) LR projector 20x lebih rendah (2.5e-7)	Rendah	Sangat Tinggi
P1	Weight decay 0.1 + grad clip 5.0	Rendah	Tinggi
P1	Skip V-ORPO (data terlalu sedikit)	Rendah	Tinggi
P2	Data augmentation (crop, rotate)	Rendah	Moderat
P2	Synthetic ORPO dari SFT data	Moderat	Moderat
P3	Semi-automated data	Tinggi	Tinggi (jangka panjang)

Prioritas	Fix	Effort	Impact
generation			
Prediksi: Dengan fix P0-P1, text-only loss seharusnya tidak naik (atau naik <5%) sambil multimodal loss tetap turun. Vision model akan menjadi “text-capable multimodal” bukan “multimodal yang lupa text”. Apakah kamu mau saya bantu draft modifikasi lengkap untuk working-molab-v6-vision-unsloth.py yang sudah include semua fix di atas?			